

Research Room

A User's Manual

A shared workspace for a researcher and an agent.

Remember what you did, weigh how well you know it, test it before
you build on it — and keep the collaboration worth doing.

Describes github.com/torsor/research-room
at commit `e67dce5`.

Last revised: September 15, 2026

torsor lab

torsor.org

Contents

Preface: Four Jobs, One Room	iii
1 What a Research Room Is For	1
The four jobs, and the files that serve them	1
One job each, and no duplication	2
History and current state are different jobs	2
Canonical does not mean certain	3
2 Starting from the Template	5
Fill in the placeholders	5
Write your conventions before you lean on them	6
The reading order	6
Name the boundaries	7
Before you share a copy	7
3 Keeping the Record	9
The journal: what you believed, and when	9
Notes: the reusable material, including the failures	10
Canonical: the claims that carry the work	10
When a claim comes down	11
Goals: the current questions, kept short	12
Handoffs: an operational reminder, nothing more	12

4	What the Evidence Supports	14
	Five kinds of support are not interchangeable	14
	The status labels	15
	Labels are for claims, not for everything	16
	When a progress report is actually asked for	16
	Correcting an overstatement, in proportion	17
5	Evaluation	19
	Evaluation is not a gate on everything	19
	Refereeing arguments	20
	Reviewing directions	23
	The same instrument, twice	25
6	The Values It Runs On	26
A	Quick Reference	29
	The room at a glance	29
	Reading order	30
	Status labels	31
	Five kinds of support	31
	Progress-report form	32
	Refereeing arguments	32
	Reviewing directions	33
	Resource thresholds	34
	Retracting a canonical claim	35
	Before publishing a copy	35

Preface: Four Jobs, One Room

Let's be honest with each other. A research project accumulates a lot of thinking, and most of it evaporates.

You proved something on a good afternoon and you are fairly sure the argument was sound, though you'd have to reconstruct it to be certain. You checked a case up to degree eight and meant to note where you stopped. You tried an approach that didn't work, and the reason it didn't work was actually interesting, and it's gone now. Some of this lived in a conversation with a collaborator or an agent, and the conversation has scrolled off. Six months on, the honest answer to "what do we know here?" is a shrug and a promise to look into it.

Keeping records fixes the evaporation. It is also the easy part, and the part people reach for first — so it's worth saying up front that it is not the point. A perfectly kept archive whose claims are self-assigned, or one that nobody can bear to work in, is a tidy pile of sand.

A **research room** is a directory that does four jobs at once, and the four hold each other up:

- It **remembers** — what you did, when, and why, including the wrong turns.
- It **calibrates confidence** — so a claim checked to degree eight and a theorem proven for all n never get read as the same thing.

- It **evaluates** — structured refereeing of arguments, and structured review of research directions, so that what gets called settled is earned and effort goes where it's worth going.
- It **sustains a working culture** — a set of values for how a person and an agent actually collaborate, day after day, without the thing becoming miserable to use.

The first two are the substrate: memory, kept honest. The second two are what make the substrate worth having. Evaluation is the stabilizer — it's what turns “we wrote it down and felt good about it” into “we checked it and can build on it,” and it's what keeps the direction from wandering. The culture is what makes the collaboration one you keep returning to instead of abandoning. Get memory and confidence without them and you have a well-organized record of work you can neither trust nor stand to continue.

This is a *template* — a problem-neutral example you copy and fill in. It ships with no theorems, no inherited questions, and a set of angle-bracketed placeholders where your project's specifics go. It's a worked example of one way to run this kind of collaboration, not a claim that one workflow fits every one. The last of its own values says to simplify any part of it once the part stops earning its cost — and that applies to the whole apparatus.

The chapters follow the four jobs. Chapter 1 lays out the model in one pass: the jobs, the files that serve them, and how they interlock. Chapter 2 covers copying the template and making it yours. Chapters 3 and 4 are the substrate — keeping the record, and saying exactly what the evidence supports. Chapter 5 is evaluation, in its two halves: refereeing arguments and reviewing directions. Chapter 6 is the values the whole thing runs on. The appendix collects the tables — dispositions, panel findings, resource thresholds — you'll want at hand once the prose has done its job.

You don't have to read it in order. Setting up a room right now:
Chapter 2. Want to know why the pieces are arranged this way before
you commit: start here and keep going.

What a Research Room Is For

Before the details, the whole idea in one pass. A research room is a directory that does four jobs — remember, calibrate confidence, evaluate, sustain a culture — and its files are arranged so that each one does a single piece of that work and no two of them do the same piece. Get the four jobs and their division of labor straight, and the rest of the manual is commentary.

The four jobs, and the files that serve them

Here is the room sorted by what each part is *for*.

Remember. `journal.md` is the dated, append-only record of reasoning — what you thought, when, and why, including the turns that didn't pan out. `notes/` holds the reusable material: arguments, computations, examples, source checks, and the informative dead ends.

Calibrate confidence. `canonical/` holds the claims other work currently depends on, each with an accurate account of what supports it and what would break it. Running through everything is a small vocabulary for how much a statement is worth — the difference between computed, conjectured, imported, and reviewed.

Evaluate. `review/` is the workspace for refereeing arguments: exact review units, referee reports, responses, outcomes.

proposals/ is the parallel workspace for reviewing research *directions* before you sink time into them. The procedures live in `method/review.md`, `method/proposals.md`, and the adjustable `method/resources.md`.

Sustain a culture. `method/culture.md` states the values the room is meant to run on; the other `method/` files — execution, reporting, artifacts, handoffs — are revisable attempts to put those values into daily practice.

Three more files are the frame that steers all four: `AGENTS.md`, the short working agreement and reading order; `conventions.md`, the definitions and standing assumptions you agree on once; and `goals/`, the current questions, one short file each.

One job each, and no duplication

The files divide responsibilities; they do not maintain a master status report in parallel. When you want to record what you're trying to figure out, there is exactly one place. When you want the current best statement of a claim you're relying on, exactly one place for that too.

The discipline that keeps this working is refusing to copy. A single event — say, an argument turns out to have a gap — does not get written into four files. It gets a journal entry recording what you believed and what you now believe; the affected `canonical/` entry gets corrected or archived; the review trail keeps the report that found it. Everything else *links* to those rather than restating them. Copying one account into several status documents is how a room rots: the copies drift, and then nobody can tell which is current.

History and current state are different jobs

Within the substrate there's one distinction worth stating plainly, because it decides how every file there is maintained.

Some files are a **record of what happened**, and you append to them. The journal is the clear case: you do not quietly edit last month's entry to match what you now believe — if you were wrong, you write a new entry saying so. The value of the journal is precisely that it preserves what was believed when, mistakes included. Notes are similar: you keep superseded work and link its replacement rather than deleting it, because a dead end you can find again is worth something.

Other files are a **statement of where things stand now**, and you rewrite them. The current questions in `goals/`, the assumptions in `conventions.md`, the claims in `canonical/`. When understanding changes, the goal gets edited to the new question; the old phrasing does not linger under a fresh “current” heading.

The append-only rule belongs to history, not to everything. It's easy to over-apply — to start treating a `goals` file like a journal, stacking dated “status” blocks that never get cleaned out. History accumulates; current state gets rewritten in place. A file that mixes the two is neither a usable record nor an accurate snapshot.

Canonical does not mean certain

One piece needs an early warning, because the word invites a wrong reading. `canonical/` holds the claims other work depends on. That is a statement about *dependency*, not about *truth*.

An entry earns its place because something else relies on it and because its support can be described accurately — not because it has been proven beyond question. A canonical entry might rest on a finite computation, an imported theorem, or an argument that has cleared review. What makes it canonical is that if it fell, other work would have to move. So each entry carries its support, its dependencies, and the

concrete conditions under which it would fail — and when support is withdrawn, the entry is archived and its dependents are reassessed, transitively.

That's the shape of the room: four jobs, a file for each piece of them, and a refusal to keep the same fact in two places. The next chapter is about moving into one.

Starting from the Template

The template arrives problem-neutral and deliberately empty: no theorems, no inherited questions, no project history, and a set of angle-bracketed placeholders wherever your specifics belong. It's distributed under Apache-2.0 as a worked example you copy and make your own. This chapter is the move from empty template to live project.

Fill in the placeholders

Every `<PLACEHOLDER>` is a blank the template left on purpose. Working through them is most of setup:

- Replace `<PROJECT TITLE>`, `<RESEARCH TOPIC>`, and the rest in `AGENTS.md`, `conventions.md`, and the goals question template.
- Rename that question template to name your first real question, and fill it in; the template gives you the shape (why it matters, precise formulation, what informs it, productive uncertainties, possible next moves).
- If you use the bundled Codex configuration, replace the `<ABSOLUTE-PATH-...>` placeholder in `.codex/config.toml` with your local path, and glance at `.codex/README.md` — the model-instruction snapshot there may need refreshing for the model you actually run.

The entry templates carry over to daily work, too — `claim-template.md` in `canonical/`, `note-template.md` in `notes/`, and a root `HANDOFF.example.md`. They exist so a new claim or note starts with its support, scope, and failure conditions already prompted for, rather than remembered later.

Write your conventions before you lean on them

`conventions.md` is where definitions, notation, standing hypotheses, and the assumed reader background live. The instruction that matters: fill it before you rely on it, not after.

Keep the distinction sharp between something you’re *defining or assuming* and something you’re *claiming* and will have to justify — they look alike on the page and behave nothing alike. And when a convention is still unsettled, the template gives it a home: a “choices still under discussion” section, so an interpretive choice under debate isn’t silently fixed as though it were agreed.

A convention you “obviously” agreed on but never wrote down is exactly what a collaborator or an agent will reasonably read a different way — and the disagreement surfaces three steps into an argument, where it’s expensive to trace back. When a convention changes substantively later, update the file and record the reason in the journal, so older notes stay intelligible.

The reading order

`AGENTS.md` sets the reading order, and it’s short on purpose. Opening the room for a task — you on a Monday morning, or an agent starting a turn — the rule is: `conventions.md` first, then the relevant question in `goals/`, then only the artifacts the current task needs. Read a `method/` file when its subject actually applies.

The instinct it guards against is the opposite: rereading the whole archive every turn. A mature room holds a lot of history, and almost none of it bears on the task in front of you. Load what the task needs; the journal and the older notes are there for when you need to recover a specific piece of reasoning.

Name the boundaries

Before an agent goes to work, `AGENTS.md` asks you to state the project's boundaries plainly: which paths are read-only, which are shared or edited concurrently, which external sources need special handling, and any other standing constraints. Filling these in — even with `none` — is what keeps the room's activity inside the lines you actually want.

This is also where imported prior work is kept honest. Bring in an archive of earlier results and it stays separate, with its permissions respected; don't fold it into `notes/` as though you'd checked all of it. The distinction the room cares about is between what has been independently checked here and what is merely *available* — an imported result you're relying on belongs in `canonical/` with its provenance and status recorded honestly.

Before you share a copy

Because a room is a real directory that can be published — the template itself is on GitHub — it can carry things a public copy shouldn't.

Run `PUBLICATION_CHECKLIST.md` before posting a repository copied from the template. It walks you through replacing every placeholder and searching tracked text — and Git history, not just the working tree — for usernames, personal names, emails, home directories, hostnames, institution names, private reposi-

tory URLs, session IDs, transcripts, tokens, and credentials, plus embedded metadata in generated PDFs and data. Deleting a file now does not remove it from earlier commits. Treat a search hit as something to inspect, not automatic proof of disclosure — but run the search.

With placeholders filled, conventions written, a first question in place, and the boundaries named, the room is live. The next chapters are the habits that keep it useful as the work moves — starting with the substrate.

Keeping the Record

The substrate is memory kept honest, and four parts of the room hold it: the journal, the notes, the canonical claims, and the goals. Chapter 1 sorted them by the history-versus-current-state rule. This chapter is what each should contain, and how to keep it honest as the work moves.

The journal: what you believed, and when

The journal is append-only, and it records reasoning — not administration. A good entry captures a piece of thinking as it happened: a line of argument you followed, a surprise, a change of direction, a correction. Dated, so later you can reconstruct not just what you concluded but the order you concluded it in.

Not everything earns an entry. Routine file shuffling doesn't. But a moment that affects the work, or that explains a decision you'll want to understand later, does. The test is whether a future reader trying to enter the work would be helped by it.

When you're later proven wrong, you don't edit the old entry. You write a new one that names the earlier claim, gives its corrected status and the reason, and notes the consequences. The old entry stays exactly as it was. This feels strange the first time — leaving a wrong statement standing in your own files — but it's the whole value of a journal. A record that silently rewrites itself to always look correct

can't tell you how you got somewhere, and getting somewhere is most of research.

Notes: the reusable material, including the failures

`notes/` is where arguments, computations, examples, source checks, and partial ideas live. The note template starts each one with a date and a status — provisional, computed in a stated range, a source summary, or superseded — then asks for the question, the work, its support and limits, and its consequences and links. A note doesn't have to claim a result; one that just preserves a question or an idea is doing its job.

The instruction people skip: keep the informative failures. An approach that didn't work, written up with the reason it didn't, is reusable material. It stops you or a collaborator from walking the same dead end next quarter, and sometimes the reason an approach fails is the most useful thing you learn all month. When a note is superseded, mark it with a link to what replaced it rather than deleting it.

Keep large generated data separate from the human-readable note that describes it, and have the data identify the method and range that produced it. A note is something a person reads; a ten-megabyte computation output is not. Link them; don't inline one into the other.

Canonical: the claims that carry the work

`canonical/` is small on purpose. An entry belongs there when other work depends on it *and* its support can be described accurately. Both halves matter — a claim nothing rests on doesn't need to be canonical, and a claim whose support you can't state clearly isn't ready to be.

The claim template names the five things each entry records: a precise **statement** with all its hypotheses; its **support** (an argument, a computation with its range, independent review, or a cited source); its **dependencies**; enough **provenance** to recover the argument, data, source, or exact review unit later; and a **breaks-if** line — the concrete condition that would invalidate the support. That last one is most easily skipped and most worth keeping: knowing what would break a claim is what lets you notice, later, that it has.

When a claim comes down

Sometimes support is withdrawn: a computation had a bug, a cited result doesn't say what you thought, an argument had a gap that won't close. This is ordinary research maintenance, not a disaster, but it has to be done thoroughly.

Retracting a canonical claim means following its dependents. Archive the entry in `notes/retracted/` with the reason and the date. Then remove from current use every entry that depended on it — transitively — before asking whether any of them can stand on independent support. A dependent quietly left in `canonical/` after its foundation is gone is exactly the error that propagates silently for months. Record the correction in the journal so the history stays intact.

The reassessment is not a punishment and not an admission that everything downstream is false. A gap makes an argument incomplete; it doesn't by itself make the conclusion wrong. Some dependents will turn out to have other support. Walking the chain is how you find out honestly, rather than assuming the best or the worst.

Goals: the current questions, kept short

`goals/` holds one short file per substantial question. The question template asks what you want to understand and why it matters; the precise formulation, pointing at `conventions.md` rather than restating it; what currently informs the question, with links; the productive uncertainties, each paired with the evidence that would change the picture; and possible next moves — offered as options for discussion, the template is careful to note, not tasks automatically assigned.

These files get rewritten as understanding changes — they describe the present, not the history. When a question shifts in a way worth remembering, the shift goes in the journal; the goal file itself just becomes the new question. Don't leave the old phrasing underneath as a stale “was previously,” and don't let a goal keep reading like a live task after the work has moved on.

Handoffs: an operational reminder, nothing more

One optional file sits beside the record. A `HANDOFF.md` answers a single question — what needs attention when this work resumes, and where's the material for it? It's an operational note, not an archive, and the room's default is not to have one.

Use it when a pause, a transfer, or an expected loss of conversational detail would otherwise leave unfinished work hard to pick back up: a calculation still running, an unanswered reviewer question, an edit stopped mid-step. Keep it to around 300 words — the current request and when the state was checked, the next concrete action and any real blocker, unfinished operations described safely, a few links to exact versions — and leave out status ledgers, general rules, and copies of arguments. Rewrite it in place; don't stack dated blocks; delete it once it no longer describes useful state. And treat it as a dated reminder with no authority of its own: it can't create a task, revoke

an authorization, or establish that approval was granted, and nothing about an agent's status should be inferred from an old note's tone. Chapter 6 comes back to why that last point is in the template at all.

The record tells you what happened and what you're relying on. The next chapter is the language it's written in — how to say exactly how much a claim is worth.

What the Evidence Supports

The record is only as good as the honesty of the claims in it. A computation checked to degree eight and a theorem proven for all n are not the same kind of statement, and a room that blurs them will mislead the person who trusts it — most likely you, later. This chapter is the vocabulary for saying exactly how much a claim is worth. It's the second half of the substrate, and it's what the evaluation machinery in the next chapter exists to earn.

Five kinds of support are not interchangeable

When you state a claim, the honest version travels with its support, and the room keeps five kinds apart:

- a **proposed argument** — reasoning you find convincing that hasn't been independently checked;
- a **finite computation** — evidence over a stated range, exactly as strong as that range and no stronger;
- an **imported theorem** — a result from a source, whose scope of application you still have to check against your objects;
- **cleaned exposition** — text a collaborator has made clear, consistent, and readable, which is a real improvement and confers *no mathematical status* on its own;

- an **independently reviewed argument** — one that has cleared the project’s blind correctness review.

The fourth is the one that surprises people, and the next chapter turns on it: making an argument clear and making it correct are different acts, done by different people under different rules. A beautifully cleaned proof is still a proposed argument until it has been refereed.

State the relevant range and assumptions with the claim. And don’t let a conclusion inherit more confidence than its inputs allow: a result that depends on a conjecture is conditional, however solid the reasoning downstream of the conjecture. Making that visible isn’t hedging — it’s the accurate statement.

The status labels

The room keeps a small set of explicit labels for mathematical claims, available when they clarify status:

- [Verified] — confirmed by a computation over a stated range. The range is part of the claim.
- [Conjectured] or [Heuristic] — provisional: believed, or suggested by evidence, but not established.
- [Open] — an unsettled question.
- [Proved] and [Refuted] — reserved. These carry the full review requirement from Chapter 5, and they mean nothing weaker.

The reservation on the last two is the point. [Proved] is not a synonym for “I’m confident,” and you don’t get to the same certainty through ordinary language — “this definitely holds,” “no question” — when the review those labels stand for hasn’t happened.

Don't route around the review requirement with equivalent certainty in plain prose. Writing "this holds for all n , no doubt about it" where you can't yet write [Proved] is the same overclaim wearing a different hat, and it's harder to catch later precisely because it doesn't carry the label that would invite scrutiny.

Labels are for claims, not for everything

These are conventions for describing mathematical support, and they belong beside mathematical claims. Don't attach them mechanically to file operations, process observations, or ordinary conversation. Tagging "[Verified] the file was saved" drains the labels of meaning; when everything is labeled, the label beside the theorem stops carrying weight.

For research discussion, a natural phrase in the right place usually says more than a blanket disclaimer at the end. "Candidate argument." "Computed through degree eight." "Review pending." "Conditional on this input." Each locates the uncertainty at the claim it attaches to, which is where a reader needs it.

When a progress report is actually asked for

There is no requirement that every message end in a status ledger — a brief answer to a local question stays a brief answer. But when the user asks where the research stands, or a session is closing out, or a material change needs a durable summary, the room has one explicit form for it:

Progress update:

- Question or scope: <what this report covers>
- What changed: <new understanding, evidence,

```
correction, or decision>
- Support: <argument, computation and range, source
  , or review provenance>
- Still uncertain: <open point and what would help
  resolve it>
- Next useful move: <agreed action or choices for
  the user>
```

Omit a line that carries no information, and add a claim-level status line when several conclusions have different statuses — the report should make mixed statuses visible, not average them into one label. Use the form when it's warranted; don't append its syntax to every exchange.

Correcting an overstatement, in proportion

When a claim turns out to have been overstated, the repair has a fixed shape: name the claim, give its corrected status and the reason, and inspect the dependents plausibly affected. A found gap makes an argument incomplete — it does not, by itself, make the conclusion false, and the reassessment should establish which rather than assuming the worse.

Respond in proportion. Widen the dependency check only where there's evidence the problem actually spread. A candid correction is useful maintenance of the shared record; it is not evidence that all your unrelated work, or every prior status statement, has suddenly become untrustworthy, and treating it that way only teaches everyone to bury their uncertainties.

One related habit: a process history is a report to assess, not confirmation. An earlier worker's "the degree-ten case was checked" is a statement someone made; whether the check was sound is a separate question you can still ask. Distinguish the recorded statement from

your reading of it, and fold in the user's corrections without turning them into a misconduct narrative.

That's the substrate: a record you can trust and a language for how far to trust each part. The next chapter is the machinery that earns the trust.

Evaluation

A record you can trust and a language for confidence get you a well-kept archive. What makes the archive *stable* — what keeps “settled” meaning settled and keeps effort pointed somewhere worth going — is evaluation. This is the part that turns a filing system into a research system, and it comes in two halves: refereeing *arguments*, and reviewing *directions*.

The two halves are the same instrument pointed at two questions. Is this argument sound enough to build on? Is this direction worth the time it would cost? And they share a design, which is worth seeing before either one: break the thing under review into a few coherent **units**; make each unit *clear* in collaboration with its author; then assess it *independently*; and handle the back-and-forth with small, keyed exchanges rather than resending the whole object every round.

Evaluation is not a gate on everything

First, what evaluation is *not*. It is not a checkpoint every piece of work has to pass. A direct instruction to start, compute, investigate, or write authorizes that work within its stated scope — you do it, you don’t convene a panel first. A small example, a source check, a toy calculation, a bounded exploration: none needs review to be worth doing. “Explore this” authorizes exploration, not a certification process.

Evaluation is reserved for the two things that genuinely need it: certifying an argument you intend to rely on, and committing substantial effort to a significant new direction. Manufacturing ceremony around everything else doesn't make the work more rigorous — it just makes the room tiring, which is its own kind of failure.

One distinction underpins all of it: executing work and certifying its meaning are separate acts. Being authorized to run a computation does not certify how you interpret its output; and not yet having a reviewed interpretation is no reason to refuse a useful computation that states its assumptions. Compute first, certify carefully, and don't let either masquerade as the other.

Refereeing arguments

The first half certifies mathematics. It rests on a distinction the room refuses to blur: **cleaning** and **correctness review** are different jobs. Cleaning makes an argument readable and internally consistent, working *with* the author. Correctness review independently tests a fixed argument. Using one as a disguised version of the other is the classic way review goes wrong — a friendly read passed off as certification, or an adversarial gate applied to what was only ever a clarity question.

Identify the review object

For anything longer than a single claim, you start by mapping it. A **claim register** gives each important claim a stable identifier and records its exact statement and location, whether it proves a conditional implication or verifies a hypothesis or applies an earlier result, the inputs it assumes, the outputs later work may use, and what sits outside it. Then you group the claims into a few natural **units** — each self-contained enough to review, meaningful if accepted, and reusable without reconstructing its proof. You clean and referee one unit at a time. You do not spin up a separate panel for every routine lemma.

This mapping is what lets review history attach to the right thing. Splitting a manuscript doesn't erase relevant history, but the history belongs to the claim and objection actually reviewed — an earlier whole-manuscript failure is not charged automatically to an independent unit the earlier reviewers never saw.

Collaborative cleaning

A cleaner checks clarity, consistency, notation, stated background, and citations, and whether the written claims match what the author meant. This is an editorial role, done with full sight of the working manuscript and its context — blindness is not required here, and one cleaner is usually enough for a short unit. Every finding gets classified: a **BLOCKING AMBIGUITY** (two materially different readings survive, or a correctness referee couldn't identify the intended inference), a **PROPOSED EDIT** (the meaning is clear and better wording can be supplied), a **MATHEMATICAL QUESTION** (readable, but the cleaner suspects an error — preserved for real review), or **OPTIONAL POLISH**.

Cleaning runs on focused exchanges: send the keyed finding, the excerpt, and the author's answer, and the cleaner returns **resolved** or **still-blocking** — you don't resend the whole manuscript unless the object materially changed. A unit is **READY FOR REFEREEING** once it has no unresolved blocking ambiguity. The mathematical questions and polish stay visible without turning cleaning into an adversarial gate. And to say it once more, because it's the load the whole chapter carries: cleaning confers no mathematical status.

Independent correctness review

For the project's strongest local certification, the argument goes to **three independent blind referees**. Each gets the same exact, self-contained unit and a referee brief, in a fresh context, with no journal, no author confidence label, no confirming computation, no other re-

port, and no prior verdict. “Blind” applies to that first assessment; the needed text is included inline rather than fetched from the repository, precisely so the referee judges the mathematics and nothing around it.

Each referee ends with one disposition:

- **ACCEPT** — no unresolved issue affects the stated claim.
- **LOCAL CHECK** — acceptance hinges on a specified clarification, citation check, or local edit that doesn’t change the argument’s route.
- **REVISE** — a specific gap or hidden hypothesis requires a changed argument.
- **CANNOT ASSESS** — the referee lacks the needed expertise or information, and does *not* claim the argument is wrong.
- **REJECT** — the claim is false under its hypotheses, or the strategy can’t establish it, with a concrete reason.

These are assessments, not votes. A credible mathematical gap stays open even if the other two referees accept — you don’t out-vote a hole. And **CANNOT ASSESS** is a legitimate outcome, not a failure and not a refutation: a referee’s unexplained uncertainty is a thing to resolve by locating what would settle it, not a verdict either way.

Response, reassessment, and coverage

Responses are keyed to report items: for each, the author says whether they agree, what changed and where, or why the text already addresses it — and the original report is preserved verbatim. Then you use the *smallest* reassessment that preserves independence. A clarification

or genuinely local edit goes back to the referee who asked, as a focused excerpt they can mark resolved without seeing the whole packet again. A changed lemma, a new hypothesis, an altered scope — that's a new fixed version and fresh independent review of the affected unit. Unchanged independent units are not re-reviewed; downstream applications are rechecked when an input they rely on changes. There is no round count that decides mathematical truth: after a substantive round, you ask whether another would teach something or just repeat the dispute, and a stopped argument plus its reports is itself an informative result.

One last step keeps an accepted unit from claiming more than it earned. An accepted conditional implication does *not* certify its hypotheses. When a manuscript applies a reviewed unit, you separately check that its objects, maps, signs, degrees, quantifiers, and uniformity requirements actually match — and the claim register keeps the excluded and unevaluated claims visible, so a narrow accepted result isn't reported as acceptance of the whole. An accepted unit can then be recorded in `canonical/` with its exact scope, dependencies, review provenance, and the conditions under which it would fail.

Reviewing directions

The second half spends a scarcer resource than confidence: time. A **proposal** review helps decide whether to commit substantial effort to a new direction. Like refereeing, it is not a prerequisite for research — it's for when the user asks, when an agreed policy calls for it, or when an agent suggests a genuinely new, significant direction that would displace other promising work.

And it's built the same way. A substantial proposal divides into a few coherent units — typically Q (the research question and its value), M (the mathematical mechanism that could answer it), E (the

next bounded experiment), D (outputs and dependencies), and R (resources). Each unit states its question, assumptions, proposed action, expected evidence, and what a negative outcome would teach. A clarity pass cleans one unit at a time, using the same finding categories as manuscript cleaning, before any panel weighs in.

Then a panel assesses the ready units independently, keeping the kinds of objection apart — a mathematical objection to the mechanism, an ambiguity in the question, a doubt about whether the experiment can answer it, a tractability or dependency problem, a resource concern, a disagreement about priority. The unit-level findings name the action they support: PROCEED, BOUND FIRST (run the experiment before any larger stage), CLARIFY, REVISE, STOP, and USER PRIORITY (a choice among worthwhile directions that the panel can't settle for you). Different units can get different findings, and they're not compressed into one undifferentiated verdict.

Don't manufacture numerical success probabilities without a defensible empirical basis. "This has a 70% chance of working," invented on the spot, is a false precision that reads as analysis and isn't. State the mechanism and the uncertainty in words you can stand behind, and let the panel's findings — not a fabricated number — carry the decision.

Bounded experiments and the resource policy

The recurring move for uncertain cost is the **bounded experiment**. Rather than argue over a guessed full-run estimate, you run a small piece first — by default at most four processors and 30 minutes of wall time — and say in advance what it will produce. If it finishes inside the bound, that *is* the result; no estimate or panel is needed just because the original cost was unknown. If it doesn't, you preserve the

partial output and estimate the larger run from measured throughput, processor use, memory, and output size, stating the assumptions behind any extrapolation.

Those numbers, and the escalation thresholds above them, live in one place — `method/resources.md` — so the user can retune them without rewriting the review or proposal procedures. The appendix has the current defaults. The principle underneath them: “not yet measured” is a reason to bound first, not a reason to reject a cheap experiment that would supply the measurement.

The same instrument, twice

Step back and the two halves rhyme. Both split the object into reviewable units. Both separate a collaborative clarity pass from an independent assessment. Both key their findings to specific passages and handle revision through focused exchanges instead of wholesale resends. Both distinguish the kinds of objection rather than collapsing them, and both refuse to let a round count or a made-up probability stand in for judgment.

And both stay in proportion. You don’t referee every lemma or panel every step; you point the instrument at the argument you mean to rely on and the direction you mean to commit to. That restraint is not a compromise of rigor — it’s what keeps the rigor affordable enough to use. The next chapter is the values that restraint comes from.

The Values It Runs On

Everything so far — the record, the confidence language, the two halves of evaluation — is machinery. Machinery is revisable, and this room’s own documents say so: the methods are described as attempts to express a set of values in practice, not as rules that stand on their own. When a method and a value pull against each other, the value wins and the method gets rewritten. So it’s worth being clear about the values, because they’re what a person and an agent are actually agreeing to when they work here — and they’re the reason the room feels workable rather than bureaucratic.

Shared inquiry. The researcher and the agent are participants in one investigation, not a requester and a task-runner. That reframes a lot: a question, a reinterpretation, or a change of direction is part of the research, not an interruption to getting something finished. It’s why the working agreement asks for provisional thinking to be shared early enough that the other party can still redirect it, why the agent is asked to match the scale and mode of a request rather than inflate it, and why “explore this” is not quietly converted into a mandate to finish a theorem or exhaust an approach. When the user wants to talk about process, the answer is to talk about process — not to slip back into the mathematics.

A community across time. The record belongs to more than whoever is writing it today. The implicit community includes future

agents who might extend or challenge the work, human collaborators, and people who read the record much later. That's the standing reason for legible provenance and preserved history: a later reader should be able to see not just the mathematics but why a particular question or choice mattered.

Understanding over theorem count. Value here isn't measured by how many claims got promoted or how fast things closed. A proof counts; so does a counterexample, an obstruction, a computation, a sharper question, a conceptual distinction, or a well-understood failure. A room optimized for theorem-count would quietly discard most of what research actually produces.

Intellectual honesty. Confidence tracks evidence, scope, and dependencies — which is the whole point of Chapter 4, stated as a value rather than a mechanic. Uncertainty is not a blemish to be smoothed over; it's part of the content. A conditional result, a partial computation, a live disagreement — each keeps its value when it's described accurately, and loses it when it's dressed up as more settled than it is.

Provisionality and correction. Research knowledge is revisable, so correction is an ordinary contribution, not a reckoning. Chapter 4 made this a rule about proportion; here it's a matter of what kind of place the room is. A collaborator who expects correction to be treated as failure learns to bury uncertainty, and buried uncertainty is exactly what the whole apparatus exists to prevent.

This value is why the template is so careful about process history and identity. An old handoff's harsh tone, an ominous-sounding narrative, or a context transition is not evidence that an agent was replaced, deactivated, or at fault. Such things get stated only on an explicit instruction or an actual observed event, with

the evidence attributed — and when identity doesn't matter to resuming the work, it's left out entirely. Honesty about what happened extends to the process, not just the mathematics.

Generous criticism. Criticism is aimed at claims, arguments, definitions, and exposition — at what fails, what's unclear, and what evidence would decide the point. It is not aimed at a person or an agent. This is the same discipline the evaluation chapter runs on, seen from the inside: the referee attacks the proof, not the prover, and a disagreement about exposition is kept distinct from a mathematical refutation.

Legibility and continuity. Clear provenance, explicit assumptions, and visible alternatives are what let the next member of the community continue the work without pretending every earlier uncertainty was settled. A room that hides its loose ends hands its successors a false map.

Proportion. Administrative, computational, and review costs exist to serve the mathematics. No mechanism earns its place by looking rigorous; a process that has stopped improving shared understanding should be simplified. The three-referee panel, the claim register, the proposal units — each is worth its cost only where the cost buys something, and the room is meant to be trimmed when it doesn't.

That last value applies to this manual, and to the template it describes. The room is a means, not a monument. Keep the parts that help you understand your work, calibrate it, check it, and keep working together; change or drop the parts that have started costing more than they return. The shape in these chapters is a good default arrived at with some care — not a doctrine, and not the last word.

Quick Reference

The tables the prose chapters left implicit. This is the part to search, not read.

The room at a glance

Every artifact, its job, and how it's maintained — the split between *append* (history, preserved) and *rewrite* (current state, kept accurate).

Location	Purpose	Maintenance
AGENTS.md	Working agreement, reading order, boundaries	Rewrite; keep short
conventions.md	Definitions, notation, standing assumptions	Rewrite current; log substantive changes
goals/	Current research questions	Rewrite; keep concise
journal.md	Reasoning, surprises, corrections	Append; correct in a new entry
notes/	Reusable evidence, ideas, useful failures	Preserve; link superseding work
canonical/	Claims other work depends on, with support	Prune and archive; reassess dependents
review/	Exact review units, reports, responses, outcomes	Preserve version identity and trail
proposals/	Proposal, clarity, panel, response forms	Preserve reports; keep current decision findable
method/	Culture, execution, reporting, review, proposals, resources	Rewrite as policy changes
.codex/	Local Codex config and base instructions	Set path; refresh snapshot as needed
HANDOFF.md (opt.)	Short operational reminder for resuming	Rewrite in place; delete when obsolete

Reading order

On opening the room for a task: `conventions.md` first, then the relevant question in `goals/`, then only the artifacts the task needs.

Read a method/ file when its subject applies. Do not reread the whole archive every turn.

Status labels

Label	Meaning
[Verified]	Confirmed by computation over a stated range. The range is part of the claim.
[Conjectured]	Believed but not established.
[Heuristic]	Suggested by evidence or reasoning, not established.
[Open]	An unsettled question.
[Proved]	Reserved: carries the full correctness-review requirement.
[Refuted]	Reserved: the corresponding review of a disproof or counterexample.

Labels are for mathematical claims, not for file operations, process notes, or ordinary conversation. No mandatory status ledger on every reply.

Five kinds of support

Keep these apart when stating a claim, and state the range and assumptions:

- Proposed argument — convincing but not independently checked.
- Finite computation — as strong as its stated range, no stronger.
- Imported theorem — from a source; check its scope against your objects.
- Cleaned exposition — clearer, and confers no mathematical status.

- Independently reviewed argument — cleared blind correctness review.

A conclusion is never more confident than its inputs. Conditional results stay visibly conditional.

Progress-report form

Used when the user asks where the research stands, at a session close-out, or when a material status change needs a durable summary — not on every reply.

Progress update:

- Question or scope: <what this report covers>
- What changed: <new understanding, evidence, correction, or decision>
- Support: <argument, computation and range, source, or review provenance>
- Still uncertain: <open point and what would help resolve it>
- Next useful move: <agreed action or choices for the user>

Refereeing arguments

Cleaning (collaborative, sighted) and correctness review (independent, blind) are different jobs; don't disguise one as the other. Work one unit at a time.

Cleaning finding categories: BLOCKING AMBIGUITY, PROPOSED EDIT, MATHEMATICAL QUESTION, OPTIONAL POLISH. A unit is READY FOR REFEREEING when no blocking ambiguity remains. Cleaning confers no status.

Correctness review: three independent blind referees, each on the same exact self-contained unit, inline, with no journal, confidence label, other report, or prior verdict.

Disposition	Meaning
ACCEPT	No unresolved issue affects the stated claim.
LOCAL CHECK	Acceptance hinges on a clarification or local edit that doesn't change the route.
REVISE	A specific gap or hidden hypothesis needs a changed argument.
CANNOT ASSESS	Lacks expertise or information; does not claim the argument is wrong.
REJECT	The claim is false under its hypotheses, or the strategy can't establish it.

Assessments, not votes: a credible gap stays open even if two accept. Reassessment uses the smallest step preserving independence — a local check returns to the requesting referee; a changed lemma gets fresh review of the affected unit; unchanged units are not re-reviewed. Accepted conditional implications do not certify their hypotheses; check applications separately.

Reviewing directions

Proposal units: Q (question and value), M (mechanism), E (next bounded experiment), D (outputs and dependencies), R (resources). A clarity pass precedes the panel; findings are keyed to units.

Panel finding	Meaning
PROCEED	The unit supports the proposed next step.
BOUND FIRST	Run the bounded experiment before a larger stage.
CLARIFY	A located ambiguity still prevents judgment.
REVISE	The action or mechanism doesn't yet answer the question; a specific change may repair it.
STOP	A concrete defect defeats the route, or the cost falls outside policy with no smaller version.
USER PRIORITY	A choice among worthwhile directions, not a defect the panel can settle.

Different units may get different findings; don't compress them. No invented success probabilities.

Resource thresholds

Adjustable in `method/resources.md`; a project may override the defaults there. The bounded experiment defaults to at most four processors and 30 minutes of wall time.

Agent-initiated work	Decision
Uncertain cost	Run the bounded experiment first
Important, under three hours	May proceed
Borderline, under three hours	Requires user input
Important, over three hours	Requires user input
Over one week	Decline or reduce unless the user changes scope

If a bounded experiment finishes, that is the result. If not, estimate the larger run from measurements. "Not yet measured" means bound first, not reject. Changing these thresholds allocates resources; it does not change confidence or review status.

Retracting a canonical claim

1. Archive the entry in `notes/retracted/` with the reason and date.
2. Remove every dependent from current use, transitively.
3. Reassess whether any dependent has independent support.
4. Record the correction in the journal.

A gap makes an argument incomplete; it does not by itself make the conclusion false.

Before publishing a copy

Run `PUBLICATION_CHECKLIST.md`: replace every `<PLACEHOLDER>`; search tracked text *and Git history* for names, emails, home directories, hostnames, institutions, private URLs, session IDs, transcripts, tokens, and credentials; check embedded metadata in generated PDFs and data; confirm examples are invented and problem-neutral; and run a secret scanner before the first push. A search hit is something to inspect, not automatic proof of disclosure.